

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	P. Sai Charan Tej	Reg. No.:	192465048
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Q1. Given an array where each element represents the maximum number of steps that can be taken forward, determine the minimum number of jumps required to reach the last index. The goal is to minimize the number of moves while ensuring reachability. Use Dynamic Programming or Greedy techniques to compute the optimal solution. Input Format: n = number of elements arr[] = jump values Sample Input: n = 5 arr = 2 3 1 1 4 Expected Output: Min Jumps = 2

Code of Conduct

I P . Sai Charan Tej(192465048)__(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Algorithm: Minimum Number of Jumps — Greedy

Step 1: Read n , the number of elements.

Step 2: Read the array $\text{arr}[]$.

Step 3: If $n \leq 1$, print $\text{Min Jumps} = 0$ and stop.

Step 4: Initialize:

- $\text{jumps} = 0$
- $\text{current_end} = 0$
- $\text{farthest} = 0$

Step 5: Traverse the array from index 0 to $n-2$.

Step 6: For each index i , calculate the farthest position reachable:

$\text{farthest} = \max(\text{farthest}, i + \text{arr}[i])$

Step 7: When $i == \text{current_end}$, one jump is required:

- Increment jumps.
- Set $\text{current_end} = \text{farthest}$.

Step 8: If $\text{current_end} \geq n-1$, the last index has been reached. Stop.

Step 9: Print the minimum number of jumps.

Example

For:

$\text{arr} = [2, 3, 1, 1, 4]$

The optimal path is:

$0 \rightarrow 1 \rightarrow 4$

Therefore:

$\text{Min Jumps} = 2$

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Technique: Greedy Algorithm

Output:



SIMATS Online Programming Lab
DAA PROGRAMMING

POTHURAJU SAI CHARAN TEJ
192465048RC

QUESTIONS

Q1
Minimum Number of Jumps
5 marks

1/1 answered

Q1 Minimum Number of Jumps5 marks

Given an array where each element represents the maximum number of steps that can be taken forward, determine the minimum number of jumps required to reach the last index. The goal is to minimize the number of moves while ensuring reachability. Use Dynamic Programming or Greedy techniques to compute the optimal solution. Input Format: n = number of elements arr[] = jump values Sample Input: n = 5 arr = 2 3 1 1 4 Expected Output: Min Jumps = 2

Your Code

```
1 n = int(input())
2 arr = list(map(int, input().split()))
3
4 if n <= 1:
5     print("Min Jumps = 0")
6 else:
7     jumps = 0
8     current_end = 0
9     farthest = 0
10
11 for i in range(n - 1):
12     farthest = max(farthest, i + arr[i])
```

Input

5
2 3 1 1 4

Output

Min Jumps = 2